

Sistemas Operativos

1er Recuperatorio 2° Parcial 1C2025

Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1)

	Part. fijo	Part. Dinamico	Segmentación
Compactacion	No necesita	Necesita, tiene alto overhead	Necesita, tiene alto overhead
Frag interna	Tiene	No tiene	No tiene
Frag externa	No tiene	Tiene	Tiene

2) Sí, se podría reconstruir. Se tendría que recorrer todos los inodos ocupados, y relevar todos los bloques en uso, actualizando en un nuevo bitmap de bloques (inicializado todo en cero).

3) Bit presencia: leído por la MMU al hacer una traducción. Modificado por el SO para indicar cuando una página es traída/quitada de/hacia memoria

Bit Uso: leído por el SO al ejecutar el algoritmo de Clock en búsqueda de una víctima. Modificado por el SO como parte del algoritmo Clock.

Bit Modificación: leído por el SO para saber si debe escribir la página en disco en caso de removerla. Escrito por la MMU al realizarse una escritura de una página de parte del Proceso.

4)

A) Falso. Es una syscall donde el SO valida los permisos preventivamente. No ocurre ninguna interrupción.

B) Verdadero. De lo contrario, podría existir un problema de seguridad al tener permisos distintos (o se puede considerar Falsa si se asume que se ignoran los permisos del soft link)

5) Si, tendría sentido, porque permitiría tener un mayor nivel de multiprogramación en el sistema, teniendo más procesos activos (con menos páginas en ram) que un esquema con pura memoria real.

Práctica

1) Primero calculamos el tamaño de la primera partición:

- El disco es de 16 GiB
- Como en la segunda partición el máximo espacio direccionable real es de 8 GiB, y hay 2gb que no se pueden direccionar, se concluye que la segunda partición es de 10 GiB.
- Por lo tanto, la primera partición es de 6 GiB

a) Tamaño máximo teórico de archivo: $2^{16} \times 64\text{KiB} = \mathbf{4GiB}$

Tamaño máximo real de archivo el mínimo de 6GiB de disco y 4GiB teórico = **4GiB**

b) Tamaño máximo teórico del fs: $2^{16} \times 64\text{KiB} = \mathbf{4GiB}$

Tamaño máximo real de fs el mínimo de 6GiB de disco y 4GiB teórico = **4GiB**

c) Tamaño FAT = $2^{16} \times 2 \text{ bytes} = 2^{17} \text{ bytes} = \mathbf{128 \text{ KiB}}$

d) El peor caso posible es recorrer toda la tabla, donde todos los clusters estuvieran ocupados, hasta llegar a la última entrada con un valor de "disponible". Entonces, serían **$2^{16} - 1$ accesos.**

2) Para saber de cuántos frames dispone la asignación fija por proceso, nos basamos en la nota que aclara que se destinan 48 KiB en ram por proceso. Teniendo 30 bits de DL y 16 bits para direccionar páginas (65536 máx), nos quedan páginas/frames de 16 KiB (14 bits de offset), por lo que disponemos de 3 frames para el proceso.

	0	1	7	2	3	2	7	1	0	3	0	
Fr. 1	0	0	0	2	2	2	2	2	0	0	0	
Fr. 2		1	1	1	3	3	3	1	1	1	1	
Fr. 3			7	7	7	7	7	7	7	3	3	Total
PFs	x	x	x	x	x			x	x	x		8 PF

3) a) Sabiendo que cada bloque es de 1 KiB y se accede a los últimos 3 punteros del inodo (el último puntero directo, el puntero indirecto simple y el doble) sabemos que se accede desde el bloque 4 hasta el 1541.

$$\text{Ptrs} \times \text{bloque} = 2^{10} \text{ B} / 2^2 \text{ B} = 2^8 = 256$$

Para leer esos 1538 bloques, 1 se accede de forma directa por el puntero directo.

Los 256 que siguen se acceden a través del bloque direccionado por el puntero indirecto simple

Los 1281 restantes se acceden a través del bloque direccionado por el puntero indirecto doble, del cuál se requerirán también $1281/256 = 5,003... = 6$ bloques de punteros

$$\text{En total} = 1538 \text{ BD} + 7 \text{ BPs} + 1 \text{ BPd} = \mathbf{1546 \text{ bloques}}$$

b) Para que la operación se realice correctamente se debe compactar primero ya que no hay espacio libre suficiente al final de la partición del proceso. Esto puede realizarse de distintas maneras, pero vamos a asumir que se mueve el Proceso 2 solamente, quedando las particiones de la siguiente manera:

Proceso 1 8 MiB	Proceso 2 5634 KiB	Libre 510 KiB	Proceso 3 2 MiB
--------------------	-----------------------	------------------	--------------------

El contenido estaría almacenado a partir de la DL 4096×2^{10} del proceso 2, teniendo éste ahora como base el byte 8192×2^{10} , queda entonces:

$$\text{DF} = \text{base} + \text{desplazamiento} = (8192 + 4096) \times 2^{10} = \mathbf{12.288 \times 2^{10} = 12.582.912}$$